

Bahan Ajar Pert 10

BAHAN AJAR - PERTEMUAN 10

Perulangan dalam Pseudocode

TP	BK 1.4 — Notasi Algoritma
----	---------------------------

A. KONSEP PERULANGAN

Perulangan (looping) adalah struktur algoritma yang menjalankan blok kode **berulang kali** selama kondisi tertentu terpenuhi.

Kenapa Perulangan Penting?

Tanpa Perulangan	Dengan Perulangan
OUTPUT 1	FOR i ← 1 TO 100
OUTPUT 2	OUTPUT i
OUTPUT 3	ENDFOR
... (100 baris)	(4 baris)

Perulangan membuat kode lebih pendek, efisien, dan mudah dipelihara!

B. FOR — PERULANGAN DENGAN COUNTER

FOR digunakan ketika **jumlah perulangan sudah diketahui** sejak awal.

Sintaks

```
FOR variabel ← nilai_awal TO nilai_akhir
  {blok kode yang diulang}
ENDFOR
```

Variabel akan secara otomatis: - **Bertambah 1** setiap kali selesai satu iterasi - Berhenti ketika melebihi nilai_akhir

Contoh 1: Mencetak 1 sampai 5

```
FOR i ← 1 TO 5
  OUTPUT i
ENDFOR
```

Iterasi	i	Output
1	1	1
2	2	2

Iterasi	i	Output
3	3	3
4	4	4
5	5	5

Contoh 2: Mencetak Bilangan Genap 2-10

```
FOR i ← 2 TO 10 STEP 2
  OUTPUT i
ENDFOR
```

Output: 2, 4, 6, 8, 10

Contoh 3: Menjumlahkan 1+2+3+...+10

```
total ← 0
FOR i ← 1 TO 10
  total ← total + i
ENDFOR
OUTPUT total
```

Iterasi	i	total
0	—	0
1	1	1
2	2	3
3	3	6
...	...	...
10	10	55

Output: 55

Contoh 4: Tabel Perkalian 5

```
FOR i ← 1 TO 10
  hasil ← 5 × i
  OUTPUT "5 × ", i, " = ", hasil
ENDFOR
```

Output:

```
5 × 1 = 5
5 × 2 = 10
5 × 3 = 15
...
5 × 10 = 50
```

C. WHILE — PERULANGAN BERSYARAT

WHILE digunakan ketika **kondisi** yang menentukan perulangan, bukan counter. Perulangan berjalan **selama** kondisi TRUE.

Sintaks

```
WHILE kondisi TRUE
    {blok kode yang diulang}
ENDWHILE
```

Contoh 1: Mencetak 1 sampai 5 (dengan WHILE)

```
i ← 1
WHILE i <= 5
    OUTPUT i
    i ← i + 1
ENDWHILE
```

Catatan: Dengan WHILE, kita harus **mengelola counter sendiri** ($i \leftarrow i + 1$). Jika lupa, akan terjadi **infinite loop**!

Contoh 2: Input Sampai 0

```
total ← 0
INPUT x
WHILE x ≠ 0
    total ← total + x
    INPUT x
ENDWHILE
OUTPUT "Total: ", total
```

Simulasi: Input: 5, 3, 2, 0 | Langkah | x | total | Kondisi $x \neq 0$? | |—|—|—|—| | 1 | 5 | 5 | TRUE | | 2 | 3 | 8 | TRUE | | 3 | 2 | 10 | TRUE | | 4 | 0 | — | FALSE → STOP |

Output: Total: 10

Contoh 3: Tebak Angka

```
angka_rahasia ← 7
tebakan ← 0
WHILE tebakn ≠ angka_rahasia
    INPUT tebakn
    IF tebakn < angka_rahasia THEN
        OUTPUT "Terlalu kecil!"
    ELSE
        IF tebakn > angka_rahasia THEN
            OUTPUT "Terlalu besar!"
        ENDIF
    ENDIF
ENDWHILE
OUTPUT "Selamat! Tebakan benar!"
```

D. REPEAT-UNTIL — MINIMAL SEKALI

REPEAT-UNTIL mirip WHILE, tapi **kondisi diperiksa di akhir**. Ini menjamin kode dijalankan **minimal 1 kali**.

Sintaks

```
REPEAT
    {blok kode yang diulang}
UNTIL kondisi TRUE
```

Contoh 1: Mencetak 1 sampai 5

```
i ← 1
REPEAT
    OUTPUT i
    i ← i + 1
UNTIL i > 5
```

Contoh 2: Menu Program

```
REPEAT
    OUTPUT "=== MENU ==="
    OUTPUT "1. Tambah"
    OUTPUT "2. Kurang"
    OUTPUT "3. Keluar"
    INPUT pilihan
    {proses sesuai pilihan}
UNTIL pilihan = 3
```

E. STUDI KASUS

Kasus 1: Faktorial $n!$

Rumus: $n! = n \times (n-1) \times (n-2) \times \dots \times 1$ **Contoh:** $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Dengan FOR:

```
PROGRAM faktorial_for
    INPUT n
    hasil ← 1
    FOR i ← 1 TO n
        hasil ← hasil × i
    ENDFOR
    OUTPUT n, "! = ", hasil
END
```

Dengan WHILE:

```

PROGRAM faktorial_while
  INPUT n
  hasil ← 1
  i ← 1
  WHILE i <= n
    hasil ← hasil × i
    i ← i + 1
  ENDWHILE
  OUTPUT n, "! = ", hasil
END

```

Uji: $n=5 \rightarrow 1 \times 2 \times 3 \times 4 \times 5 = 120$

Kasus 2: Deret Fibonacci

Rumus: $F_0=0, F_1=1, F_n = F_{n-1} + F_{n-2}$ **Deret:** 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

```

PROGRAM fibonacci
  INPUT n
  a ← 0      {F0}
  b ← 1      {F1}
  OUTPUT a
  OUTPUT b
  FOR i ← 3 TO n
    c ← a + b
    OUTPUT c
    a ← b
    b ← c
  ENDFOR
END

```

Uji: $n=10 \rightarrow 0, 1, 1, 2, 3, 5, 8, 13, 21, 34$

Kasus 3: Bilangan Prima

Bilangan prima hanya habis dibagi 1 dan dirinya sendiri.

```

PROGRAM cek_prima
  INPUT n
  IF n < 2 THEN
    OUTPUT "Bukan prima"
  ELSE
    prima ← TRUE
    FOR i ← 2 TO n-1
      IF n MOD i = 0 THEN
        prima ← FALSE
      ENDIF
    ENDFOR
    IF prima = TRUE THEN
      OUTPUT n, " adalah prima"
    ELSE
      OUTPUT n, " bukan prima"
    ENDIF
  ENDIF
END

```

Uji: $n=7 \rightarrow$ prima. $n=10 \rightarrow$ bukan prima.

F. PERBANDINGAN FOR vs WHILE vs REPEAT-UNTIL

Aspek	FOR	WHILE	REPEAT-UNTIL
Counter otomatis		x manual	x manual
Cek kondisi	Di awal (implisit)	Di awal	Di akhir
Minimal eksekusi	0	0	1
Risiko infinite loop	Rendah	Tinggi (jika lupa update)	Tinggi
Kapan pakai	Jumlah pasti	Kondisi berubah	Minimal 1x

Pilihan Tepat:

Situasi	Pakai
Cetak 1-100	FOR
Input sampai user ketik 0	WHILE
Menu program (muncul minimal sekali)	REPEAT-UNTIL
Jumlah perulangan tidak pasti	WHILE / REPEAT-UNTIL

G. KESALAHAN UMUM

Kesalahan	Salah	Benar
Infinite loop (lupa update)	<code>WHILE x<10</code> tanpa <code>x←x+1</code>	Tambahkan <code>x←x+1</code>
FOR dengan STEP lupa	<code>FOR i←1 TO 10 STEP 2</code> untuk genap	(STEP 2 benar untuk loncat 2)
REPEAT-UNTIL logika terbalik	<code>UNTIL x=0</code> padahal mau stop	<code>UNTIL x=0</code> (stop jika 0)
WHILE vs IF bingung	WHILE dipakai untuk sekali cek	IF untuk sekali, WHILE untuk ulang